PROJECT 3 REPORT

Automated Multi-Tier Application Deployment

Spring 2026

In partial fulfillment
Of the requirement for the course of
Cloud Computing

SUBMITTED BY

Muhammad Bilal Tariq (22i-1297)

Muhammad Azeem Ashfaq (22I-1057)

Date: April 26, 2026

re

DEPARTMENT OF COMPUTER SCIENCE, FAST-NU, ISLAMABAD

1. Introduction

This report documents the complete design, implementation, and deployment of an automated multi-tier microservices application on Amazon Web Services (AWS). The project demonstrates end-to-end DevOps practices including containerization, infrastructure as code, configuration management, container orchestration, and continuous integration and deployment (CI/CD).

The application chosen for deployment is Google's Online Boutique — an open-source, cloud-native microservices demo e-commerce application. It consists of 11 independently deployable microservices written in multiple programming languages including Go, Python, Node.js, C#, and Java.

1.1 Project Objectives

- Deploy a microservices application on AWS EC2 using industry-standard DevOps tools
- Containerize all 11 microservices using custom Dockerfiles with 2-stage builds
- Provision AWS infrastructure using Terraform (Infrastructure as Code)
- Configure the server automatically using Ansible (Configuration as Code)
- Orchestrate containers using Kubernetes (microk8s)
- Implement a CI/CD pipeline using GitHub Actions and ArgoCD

1.2 Application Overview

The Online Boutique is a microservices-based e-commerce web application consisting of the following services:

Service	Language	Purpose	Port
frontend	Go	Web UI serving the storefront	8080
cartservice	C# / .NET	Manages shopping cart using Redis	8080
productcatalogservice	Go	Serves product listings from JSON	3550

Service	Language	Purpose	Port
currencyservice	Node.js	Currency conversion via gRPC	7000
paymentservice	Node.js	Processes payment transactions	50051
shippingservice	Go	Calculates shipping costs	50051
emailservice	Python	Sends order confirmation emails	8080
checkoutservice	Go	Orchestrates the checkout flow	5050
recommendationservice	Python	Suggests related products	8080
adservice	Java	Serves targeted advertisements	9555
loadgenerator	Python (Locust)	Simulates user traffic	-

2. System Architecture

The deployment follows a layered architecture where each tool serves a specific role in the pipeline. The diagram below illustrates the complete flow from developer to production:

2.1 Infrastructure Architecture

2.2 AWS Networking

The AWS infrastructure is organized into a Virtual Private Cloud (VPC) with the following components:

Component	Configuration	Purpose
VPC	CIDR: 10.0.0.0/16	Isolated private network on AWS
Public Subnet	CIDR: 10.0.1.0/24	Hosts the EC2 instance with public IP
Internet Gateway	Attached to VPC	Allows internet traffic in and out
Route Table	0.0.0.0/0 -> IGW	Routes all traffic to internet gateway
Security Group	Ports: 22,80,443,8080,9090,30000,30001	Firewall rules for EC2 instance
EC2 Instance	t3.medium, Ubuntu 22.04, 30GB SSD	Runs the entire application

3. Containerization (Docker)

Each microservice has a custom Dockerfile implementing a multi-stage build pattern. This pattern significantly reduces image size and improves security by excluding build tools from the final image.

3.1 Multi-Stage Build Pattern

Every Dockerfile follows a two-stage pattern:

- Stage 1 (Builder): Uses a full SDK/compiler image to build the application binary
- Stage 2 (Runner): Copies only the compiled output into a minimal base image

Example — cartservice Dockerfile (C# / .NET):

```
# Stage 1: Build
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS builder
WORKDIR /app
COPY cartservice.csproj .
RUN dotnet restore cartservice.csproj
COPY . .
RUN dotnet publish cartservice.csproj -c Release -o /app/publish --no-restore

# Stage 2: Run (much smaller image)
FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=builder /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "cartservice.dll"]
```

3.2 Base Images Used

Service	Build Image	Runtime Image	Size Reduction
frontend	golang:1.26-alpine	alpine:3.21	~90%
cartservice	dotnet/sdk:10.0	dotnet/aspnet:10.0	~70%
adservice	gradle:8.6-jdk21	eclipse-temurin:21-jre	~75%
emailservice	python:3.12-slim	python:3.12-slim	Optimized deps
currencyservice	node:22-alpine	node:22-alpine	Prod deps only

3.3 CI Integration

Docker images are not built manually. Instead, GitHub Actions automatically builds and pushes all 11 images to DockerHub on every commit to the main branch. Each image is tagged with the git commit SHA for traceability, ensuring every deployment is reproducible.

4. Infrastructure as Code (Terraform)

Terraform is used to provision all AWS resources declaratively. The entire infrastructure can be created with a single command and destroyed just as easily — eliminating manual console clicks and ensuring reproducible environments.

4.1 File Structure

File	Purpose
providers.tf	Configures the AWS provider (region, credentials)
variables.tf	Defines configurable inputs (region, instance type, AMI, key path)
main.tf	Defines all AWS resources (VPC, subnet, security group, EC2)
outputs.tf	Prints EC2 IP, DNS, SSH command, and app URL after apply
.gitignore	Prevents sensitive .tfstate files from being committed to GitHub

4.2 Resources Provisioned

Running terraform apply creates the following 8 resources on AWS:

- aws_vpc — Virtual Private Cloud (10.0.0.0/16)
- aws_internet_gateway — Internet access for the VPC
- aws_subnet — Public subnet (10.0.1.0/24) in us-east-1a
- aws_route_table — Routes traffic to the internet gateway
- aws_route_table_association — Links subnet to route table
- aws_security_group — Firewall with ports 22, 80, 443, 8080, 9090, 30000, 30001
- aws_key_pair — SSH public key for server access
- aws_instance — EC2 t3.medium with Ubuntu 22.04 and 30GB SSD

4.3 Deployment Commands

```
# Initialize (download AWS provider plugin)
terraform init

# Preview what will be created
terraform plan

# Create all resources on AWS
terraform apply

# Destroy all resources (after project is complete)
terraform destroy
```

4.4 Terraform Output

After successful deployment, Terraform prints the server connection details:

```
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:
  ec2_public_ip  = "18.232.178.157"
  app_url        = "http://18.232.178.157"
  ssh_command    = "ssh -i ~/.ssh/id_rsa ubuntu@18.232.178.157"
```

5. Configuration as Code (Ansible)

Ansible automates the configuration of the EC2 server. Instead of manually SSHing into the server and running installation commands, Ansible executes everything automatically from the local machine over SSH.

5.1 Playbook Structure

```
ansible/
├── inventory.ini        # Server address + SSH credentials
├── playbook.yml         # Master file that runs all roles
└── roles/
    ├── docker/
    │   └── tasks/
    │       └── main.yml # 10 tasks to install Docker
    └── microk8s/
        └── tasks/
            └── main.yml # 14 tasks to install Kubernetes
```

5.2 Docker Role

The Docker role installs Docker Engine on the EC2 instance with the following tasks:

- Update the apt package cache
- Install required system packages (curl, gnupg, ca-certificates)
- Add Docker's official GPG key and repository
- Install docker-ce, docker-ce-cli, and containerd.io
- Start Docker service and enable it on boot
- Add ubuntu user to the docker group (no sudo needed)

5.3 microk8s Role

The microk8s role installs and configures a local Kubernetes cluster:

- Install snapd package manager
- Install microk8s (Kubernetes 1.28/stable) via snap
- Wait for microk8s to be fully ready (up to 2 minutes)
- Enable DNS, Storage, Ingress, and Registry addons
- Configure kubectl for the ubuntu user
- Add kubectl alias for convenience

5.4 Execution

```
# Run from local machine — configures remote server automatically
ansible-playbook -i inventory.ini playbook.yml

# Expected result:
# PLAY RECAP
# 18.232.178.157 : ok=25  changed=12  unreachable=0  failed=0
```

6. Container Orchestration (Kubernetes)

Kubernetes manages the deployment and operation of all containerized microservices. microk8s is used as a lightweight, single-node Kubernetes distribution suitable for this project's single-EC2-instance deployment model.

6.1 Manifest Structure

Each microservice has two YAML manifest files:

- deployment.yml — Defines the container image, replicas, ports, environment variables, and resource limits
- service.yml — Exposes the deployment as a network service (either ClusterIP or NodePort)

6.2 Service Types

Service Type	Used By	Accessible From
NodePort	frontend (port 30000)	External internet users
ClusterIP	All other 10 services	Only inside the cluster

6.3 Resource Management

Each deployment defines resource requests and limits to ensure fair resource allocation across all 11 services on the single EC2 node:

```
resources:
  requests:
    cpu: "100m"      # 0.1 CPU core minimum
    memory: "64Mi"   # 64MB RAM minimum
  limits:
    cpu: "200m"      # 0.2 CPU core maximum
    memory: "128Mi"  # 128MB RAM maximum
```

6.4 Namespace

All microservices are deployed in a dedicated namespace called 'boutique', which isolates them from system components like ArgoCD running in their own namespaces.

6.5 Deployed Pods

After successful deployment, all pods show Running status:

```
$ microk8s kubectl get pods -n boutique
NAME                                 READY   STATUS    RESTARTS
adservice-xxx                        1/1     Running   0
cartservice-xxx                      1/1     Running   0
checkoutservice-xxx                  1/1     Running   0
currencyservice-xxx                  1/1     Running   0
emailservice-xxx                     1/1     Running   0
frontend-xxx                         1/1     Running   0
paymentservice-xxx                   1/1     Running   0
productcatalogservice-xxx            1/1     Running   0
recommendationservice-xxx            1/1     Running   0
redis-xxx                            1/1     Running   0
shippingservice-xxx                  1/1     Running   0
```

7. CI/CD Pipeline

The CI/CD pipeline consists of two components: GitHub Actions for Continuous Integration and ArgoCD for Continuous Deployment. Together they automate the entire build, push, and deploy cycle triggered by a single git push.

7.1 Pipeline Overview

```
Developer: git push to main
              |
              v
GitHub Actions (CI):
  [1] Checkout repository
  [2] Login to DockerHub
  [3] Build all 11 Docker images
  [4] Push images with commit SHA tag
  [5] Update image tags in k8s manifests
  [6] Commit and push updated manifests
              |
              v
ArgoCD (CD) — polls every 3 minutes:
  [1] Detects manifest changes in repo
  [2] Pulls new images from DockerHub
  [3] Performs rolling update on cluster
  [4] Reports Healthy + Synced status
```

7.2 GitHub Actions Workflow

The workflow file at .github/workflows/ci.yml triggers on every push to the main branch that modifies files in src/ or k8s/ directories. Key features include:

- Docker layer caching (type=gha) — reduces build time on subsequent runs
- Dual tagging — each image gets both a unique commit SHA tag and ':latest'
- Automatic manifest updates — sed replaces image tags in all deployment.yml files
- Bot commits — GitHub Actions commits the updated manifests back to the repo

7.3 Required Secrets

Secret Name	Value	Purpose
DOCKERHUB_USERNAME	DockerHub username	Identifies the image repository owner
DOCKERHUB_TOKEN	DockerHub access token	Authenticates image push operations

7.4 ArgoCD Configuration

ArgoCD is installed on the EC2 server in its own namespace and configured to watch the GitHub repository for changes to the k8s/ directory. Key configuration options:

- automated.prune: true — removes resources deleted from manifests
- automated.selfHeal: true — reverts manual changes to match the repo
- directory.recurse: true — scans all subdirectories under k8s/
- CreateNamespace: true — creates the boutique namespace if missing

7.5 Pipeline Results

After a successful pipeline run, GitHub Actions shows:

```
CI - Build and Push Docker Images ☑
Duration: 7m 50s
Triggered by: push to main
```

And ArgoCD dashboard shows:

```
APP HEALTH:   Healthy
SYNC STATUS:  Synced to HEAD
LAST SYNC:    Sync OK
Auto sync:    enabled
```

8. Results and Testing

8.1 Live Application

The Online Boutique application is successfully deployed and accessible at:

Service	URL	Status
Online Boutique App	http://18.232.178.157:8080	Live
ArgoCD Dashboard	https://18.232.178.157:9090	Live

The application displays the 'AWS' badge confirming it is running on the AWS platform. All pages load correctly including the product catalog, individual product pages, and the shopping cart.

8.2 Service Health Summary

Service	Status	Restarts	Notes
frontend	Running	0	NodePort accessible on port 8080
cartservice	Running	0	Fixed: port corrected from 7070 to 8080
productcatalogservice	Running	0	Healthy
currencyservice	Running	0	Fixed: disabled GCP profiler
paymentservice	Running	0	Fixed: disabled GCP profiler
shippingservice	Running	0	Healthy
emailservice	Running	0	Healthy
checkoutservice	Running	0	Healthy
recommendationservice	Running	0	Healthy
adservice	Running	0	Healthy
redis	Running	0	In-memory data store for cart

8.3 Issues Encountered and Resolutions

Issue	Cause	Resolution
Ansible inventory failed	Windows line endings (CRLF) in .ini file	Ran dos2unix on inventory.ini

Issue	Cause	Resolution
cartservice crash	Wrong port in manifest (7070 vs actual 8080)	Updated containerPort and PORT env var to 8080
currency/payment crash	Google Cloud Profiler requires GCP Project ID	Added DISABLE_PROFILER=1 env variable
App unreachable on port 30000	microk8s iptables FORWARD policy was DROP	Set iptables FORWARD policy to ACCEPT and made persistent
Terraform failed in WSL	Windows filesystem does not support Linux file permissions	Copied terraform files to WSL home directory (~/)
ArgoCD not deploying pods	ArgoCD only scanned top-level k8s/ directory	Added directory.recurse: true to application.yml

9. Conclusion

This project successfully demonstrates a complete, automated DevOps pipeline for deploying a multi-tier microservices application on AWS. Every component of the deployment is defined as code, making it fully reproducible and version-controlled.

The key achievements of this project include:

- Containerized 11 microservices across 5 different programming languages using multi-stage Docker builds
- Provisioned a complete AWS infrastructure including VPC, subnets, security groups, and EC2 instance using Terraform with zero manual console interaction
- Automated server configuration using Ansible, installing Docker and Kubernetes without any manual SSH commands
- Deployed all microservices to Kubernetes with proper service discovery, resource limits, and namespace isolation
- Implemented a complete CI/CD pipeline where a single git push triggers automatic image building, pushing, and deployment
- Resolved real-world deployment issues including port mismatches, cloud-specific dependencies, and networking configurations

The result is a fully functional, cloud-deployed e-commerce application that automatically updates whenever code changes are pushed to the repository — demonstrating the power of modern DevOps practices.

9.1 Deliverables Summary

Deliverable	Location	Status
Source code + Dockerfiles	src/*/Dockerfile	Complete
Terraform infrastructure code	terraform/*.tf	Complete
Ansible configuration playbooks	ansible/	Complete
Kubernetes manifests	k8s/	Complete
GitHub Actions CI workflow	.github/workflows/ci.yml	Complete
ArgoCD application config	k8s/argocd/application.yml	Complete
README documentation	README.md	Complete

9.2 Repository

All source code, infrastructure code, and configuration files are available at:

https://github.com/BilalTariq03/microservices-demo